TECHIN 513 – Lab 2

Systems, Convolution, and Time Domain Filtering

Lab Instructions

1. Please approach the course instructor or grader for assistance if you need help.
2. We highly recommend working in groups of **2 members**. When collaborating with students in other groups, please do not share code but only discuss the relevant concepts and implementation methods.
3. Please document your code well by **using appropriate comments, variable names, spacing, indentation, docstrings**, etc. Please refer to TECHIN 509 for more information.
4. The starter code is not binding on you. Feel free to modify it as you wish. Everything is fine so long as you are getting the correct results.
5. Please upload the .ipynb file to canvas. Use Markdown cells appropriately to answer the questions. Each team only needs to submit one report.
6. Please enter the names of all the team members in your Jupyter notebook so that you do not lose credit for your work.

Lab Submission Requirements

In your notebook, briefly describe your results and discuss the problems you encountered, the solutions that you came up with. Also, answer the questions asked in each section. Make sure to include the code of all tasks in code cells.

Lab Objectives

This lab is meant to give you an introduction to work with discrete time signals and systems. You will explore convolution operation. Moreover, you will implement some temporal filters in discrete time from scratch using Python.

Task 1: Applications of Convolution

In this class, we learned a new operation called convolution. Similar to addition and multiplication, convolution is a powerful and widely-used operation in various applications such as robotic control and image processing. In this task, you will explore two uses of convolution: polynomial multiplication and image processing (blurring and edge detection).

1.1 Polynomial multiplication and Convolution (2 points)

In this subtask, you will explore the relationship between convolution and polynomial multiplication. Through your exploration, see if you can make sense of tabular method for performing convolution sum. In fact, convolution is a popular method to efficiently calculate the

multiplication of two polynomials, especially when these polynomials involve large numbers of terms.

1. Consider two polynomial functions $f(x) = 1 + 2x + 3x^2$ and $g(x) = 4 + 5x + 6x^2$. Write out the expression of $f(x) \cdot g(x)$ in a Markdown cell. What are the coefficients of the resulting polynomial?
2. Extract the coefficients of $f(x)$ and $g(x)$, and store each of them into a numpy array. Perform the convolution between these two arrays using np.convolve(). Compare the result with the coefficients of $f(x) \cdot g(x)$. Are the results from steps 1 and 2 the same?

1.2 Convolution for Image Blurring and Edge Detection (3 points)

Convolution can be extended beyond 1D arrays. In this subtask, we will apply convolution to images for blurring and edge detection. To complete this subtask, we will need scipy package.

1. Import signal from the scipy package so we can use signal.convolve2d() function.
2. Use the sample code to load lenna.jpeg and convert it to a 2D numpy array (image in grayscale). Display the image for our reference.
3. Use signal.convolve2d() to perform the convolution between the 2D numpy array and smooth_kernel. Explain what is the effect of the convolution operation when using the smooth_kernel.
4. Repeat step 3 with horiz_edge_kernel. Explain the effect of convolution operation when using the horiz_edge_kernel.
5. Based on step 4, define a vert_edge_kernel to detect edges along the vertical direction.

In computer science, although the smoothen kernel and edge detection kernel are applied using convolve2d(). The actual operation slightly differs from the mathematical definition of convolution. Particularly, the kernel is not rotated in these cases. However, as the kernel is symmetric, the rotation operation can be omitted.

The smoothen operation and edge detection operation on images are essentially applying filters to the image data. The smoothen and edge detection operations can be understood as a low-pass and high-pass filter, respectively. In the remaining tasks, we will explore more about filters.

Task 2: Implementing Discrete Time Filters to Filter Time-Series Data

In this section, we are going to see how digital filters are implemented on time-series data. Just like frequency domain, the time domain tells us a lot of important information about signals and their associated phenomena. But, as you may observe in your wearable devices, these signals will not be perfect for analysis as they will almost invariably include some sort of corruption in the form of noise, outliers, etc. We are going to see some ways of dealing with some commonly occurring noise and means of corruption. This is **not an exhaustive list** but is meant to give you a head start into how you would go about analyzing time-series data.

2.1 Implementing a Running Mean Filter (3 points)

The running mean filter aims to filter out high levels of random noise in your signal. It works by replacing each point of the signal by the mean of a given number of points around the point and the point itself. The exact number of points on either side of the point in question is a parameter that is left to our discretion. Let us call this parameter k. To implement the mean filter, use the following sequence of instructions:

3. You can start by defining your k value to be 20.
4. Define a random noise signal to be added to the base signals in the starter code.
5. Now create two new noisy signals which are just the result of adding the same noise to the two different base signals.
6. With the defined value of k apply the mean filter to both the signals. To do this, loop through the signal points starting from k and replace each point by the average of k points that come before it, k points that come after it and the point itself.
7. Create two subplots, one for each signal. Each subplot must have the original signal in the first subplot, the noisy signal in the second subplot and the filtered signal overlaid on top of the noisy signal in the third subplot. Your results should look like the plot shown below. Be sure to label your plots and use a legend of the third subplot in both cases to show which one is the filtered signal, and which one is the noisy signal.

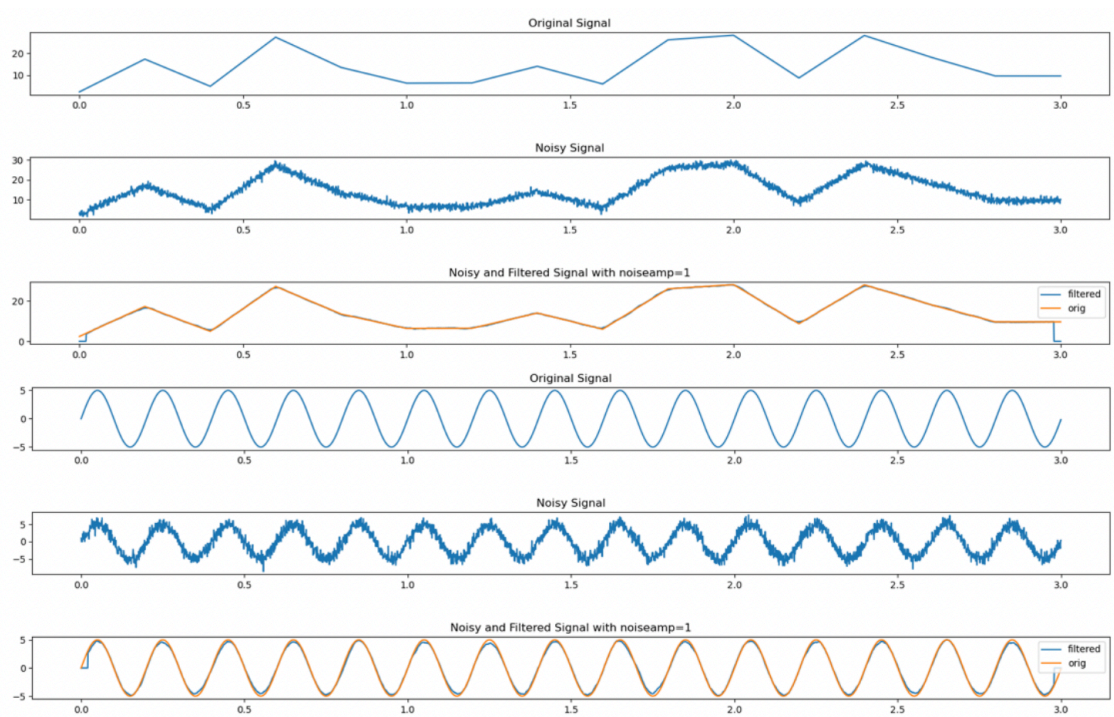

Figure 1: Expected Result for Assignment 1.1

Discussion (2 points)

1. Comment on how the results and plots change when you amplify the noise more and change the value of k.
2. Mention and explain any ONE of many possible drawbacks of the mean filter in analyzing noisy time series?

2.2 Implementing a Median filter to Remove Spikes (3 points)

The median filter is very good at dealing with outliers in data. To observe and appreciate this, a signal with noise in the form of sharp spikes has been provided. You will denoise the data or signal building on top of the starter code and following the sequence of instructions given:

1. Run the code to visualize the noisy signal. The original signal is essentially the signal you see minus the spikes, and this is evident from the code as well.
2. The spikes will be removed by applying a threshold to determine which points are indeed spikes and which points are not. To figure out what this threshold is it helps to plot a histogram of the signal to see where most of the points lie. Use plt.hist() to plot the histogram of the signal.
3. Now, from the histogram, choose a threshold that would successfully, differentiate the signal from the spikes.
4. Find all the points in the signal whose value exceeds the threshold using np.where() and store it in the variable suprathresh.
5. Now apply the median filter at only those points where the signal value exceeds the threshold (the points contained in suprathresh). To do this, take the median of k points on either side of the point of interest and the point itself. It's essentially the same as the mean filter but we are computing the median instead of the mean. (***Hint***: use np.median() to compute the median and be mindful of the bounds on the signal and the time window for the filter)
6. Plot your result by overlaying the filtered signal on top of the noisy signal in the same plot.
7. The three plots you will generate for this assignment should look similar to the ones below.

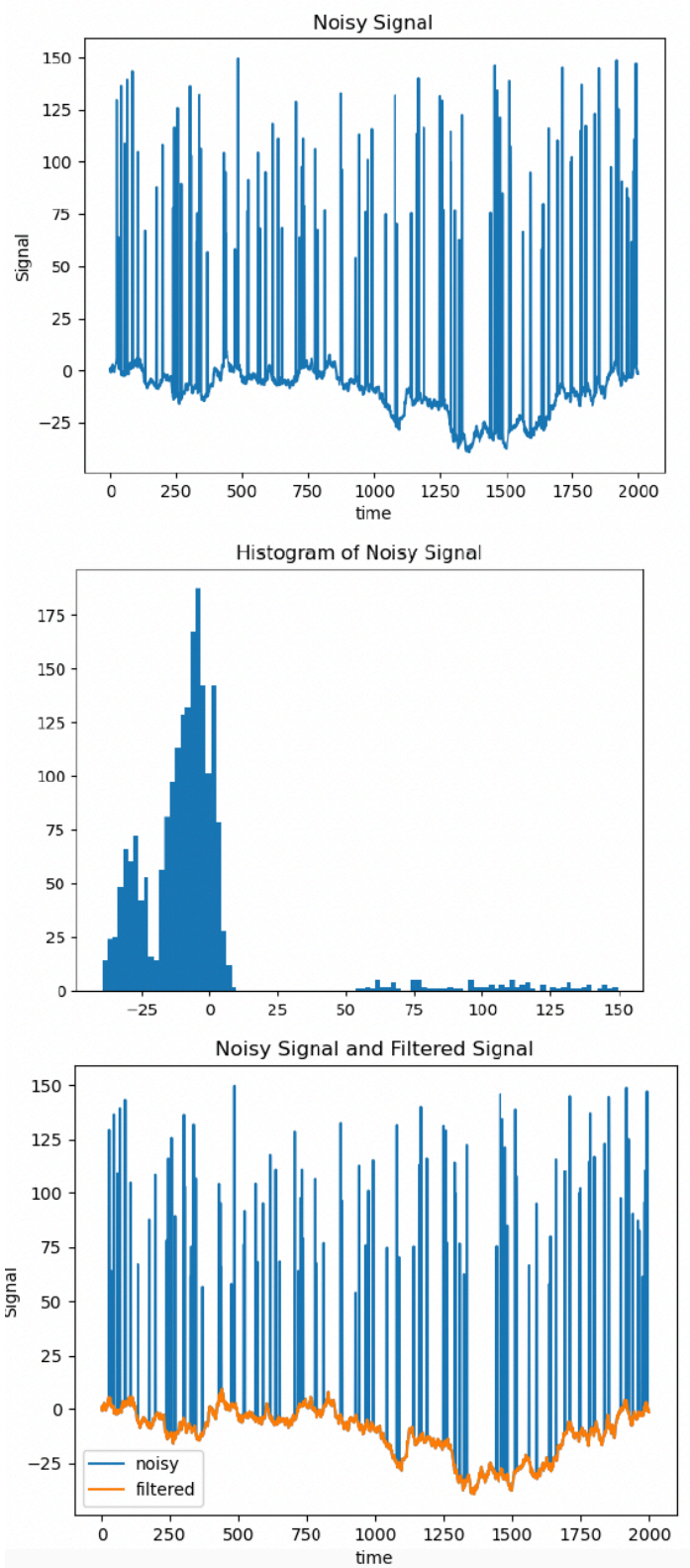

Figure 2: Expected Result for Assignment 1.2

Discussion (1 point)

1. Compare the mean and median filters in terms of their uses and one advantage and disadvantage one has over the other.

Task 3: Denoising an EMG Signal (4 points)

Electromyography (EMG) is a measure of motor unit activity of the muscles of macroorganisms such as humans, animals, birds, etc. It can be used to analyze movement, diagnose muscle problems and all kinds of other things. But when the EMG is extracted, it is not noise free (as most things aren't!). There are several features of the EMG one would be interested in. But we are just going to look at a time-domain feature in the abrupt changes as they signify muscle activity. So, checking when muscles are triggered could be of importance for many clinical reasons. We will see how the signal can be pre-processed to better lend itself to detection of spikes in activity using the Teager-Kaiser energy tracking operator (TKEO). You will denoise and pre-process the EMG signal with the following sequence of instructions:

1. The starter code will start you off with reading in the signal and extracting the variables of interest. Please build on top of that.
2. The TKEO algorithm is given as $y_t = x_t^2 - x_{t+1}x_{t-1}$. Here, let y_t be a point in the filtered signal at index t and let x be the input EMG signal and x_{t+1} and x_{t-1} are points in the EMG signal at indices one after and one before the index t respectively. Fill in a new filtered signal whose indices are updated with TKEO algorithm described.
3. Notice that the original EMG signal and the filtered signal obtained as a result of TKEO are not on the same scale or units. EMG is measured in μV or mV and the TKEO filtered signal is measured in energy. This is common, and you must be aware that this can happen.
4. There are several ways to deal with this and one such way, which will be implemented in the lab, is converting the signals to a corresponding Z-score signal. Recall from your statistics class that the Z-score is essentially how many standard deviations away from the mean the current value in question is. So, to implement this:
   - Find the mean of all the points of the signal to the left of the 0s time point as that is the base we are comparing against.
   - Find the standard deviation of the same slice of the signal from the previous part.
   - Subtract the mean calculated from the signal and divide by the standard deviation. Do this for both the original and filtered signal (obtained from TKEO in part b). This should give you two new signals, one Z-score signal corresponding to the original EMG signal and the other corresponding to the filtered signal from part b.

1. d) Plot your results showing the filtered signal overlaid on top of the original signal for both methods. Use separate plots, one for the un-scaled TKEO method and the other one for the Z-score scaled result. When plotting the un-scaled TKEO results, normalize the

signals being plotted by dividing the signal by its maximum value so that the range of values in both signals are capped at 1 and can be more easily compared. When calculating and plotting the Z-score, the original signal refers to the original signal also converted to the corresponding Z-score signal.

2. e) The expected result should look something like what is shown in Figure 3. Even if does not exactly match the result shown below, it must be clear that the spike is very distinct and is the only sign of activity with all else being near zero. It may be useful, though not required for this submission, to visualize the Z-score scalings of the original and TKEO signals separately. Make sure you understand why the Z-score scaling results in the relative magnitudes of the original and TKEO signals.

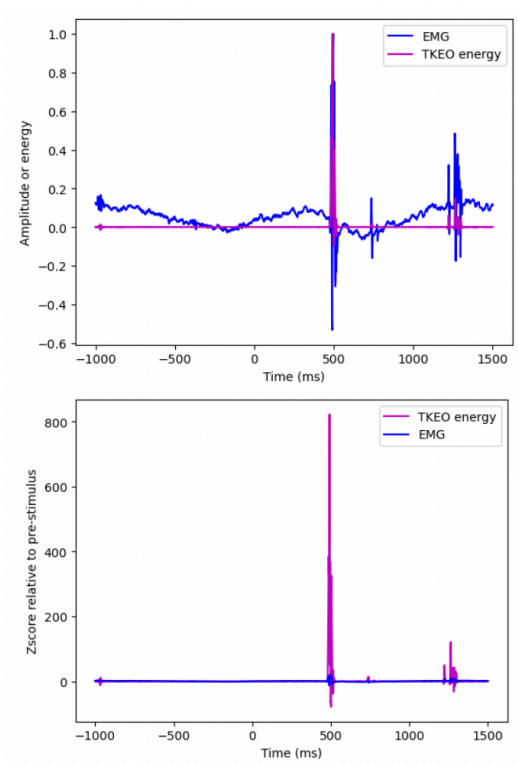

Figure 3: Expected Result for Assignment 2

Discussion (2 points)

• How would the other two filters implemented, i.e., the running mean and median filters fare against the TKEO method in analyzing the EMG signal in this fashion?

• If you had to use a running mean filter or a median filter to analyze the EMG signal to detect muscle activity, which one would you prefer and why?